

Unity Image Loader

OpenUPM v7.0.1 Asset Store View Editor Runtime

Stars 178 License MIT #StandWithUkraine

Image Loader Animation

Async image loader with two caching layers for Unity. It supports loading images from web or local paths and provides memory and disk caching to optimize performance. The package includes features for automatic image setting, cancellation handling, error handling, and lifecycle management.

Wait for image get loaded then set:

```
image.sprite = await ImageLoader.LoadSprite(imageURL);
```

Don't wait, use callback to set loaded image later:

```
ImageLoader.LoadSprite(imageURL).Consume(image).Forget();
```

Use callback to set image and still wait for the completion:

```
await ImageLoader.LoadSprite(imageURL).Consume(image);
```

Features

- ✓ Async loading from **Web** or **Local** `ImageLoader.LoadSprite(imageURL);`
- ✓ **Memory** and **Disk** caching - tries to load from memory first, then from disk
- ✓ Dedicated thread for disk operations
- ✓ Supports loading of `Texture2D` and `Sprite`
- ✓ Avoids loading same image multiple times simultaneously, a new load task waits for completion of existed task
- ✓ Uses `UnityWebRequest` to load data which works smooth across all platforms including `WebGL`
- ✓ Cache supported on at `WebGL`. Memory cache works, Disk cache isn't allowed by the platform
- ✓ Set into Image `ImageLoader.LoadSprite(imageURL).Consume(image);`
- ✓ Set into RawImage `ImageLoader.LoadSprite(imageURL).Consume(rawImage);`
- ✓ Set into Material `ImageLoader.LoadSprite(imageURL).Consume("_MainTex", material);`
- ✓ Set into SpriteRenderer `ImageLoader.LoadSprite(imageURL).Consume(spriteRenderer);`
- ✓ Set into anything
- ✓ Cancellation `ImageLoader.LoadSprite(imageURL).Cancel();`
- ✓ Cancellation callback `ImageLoader.LoadSprite(imageURL).Cancelled(() => ...);`
- ✓ Error callback `ImageLoader.LoadSprite(imageURL).Failed(exception => ...);`

- ✓ Debug level for logging `ImageLoader.settings.debugLevel = DebugLevel.Error;`
- ✓ Debug level per each task
`ImageLoader.LoadSprite(imageURL).SetLogLevel(DebugLevel.Trace);`

Stability status

Unity Version	Editor test	Player test	Build test
2019.4.40f1	 2019.4.40f1	 2019.4.40f1	 2019.4.40f1
2020.3.40f1	 2020.3.40f1	 2020.3.40f1	 2020.3.40f1
2021.3.45f1	 2021.3.45f1	 2021.3.45f1	 2021.3.45f1
2022.3.57f1	 2022.3.57f1	 2022.3.57f1	 2022.3.57f1
2023.1.20f1	 2023.1.20f1	 2023.1.20f1	 2023.1.20f1
2023.2.20f1	 2023.2.20f1	 2023.2.20f1	 2023.2.20f1
6000.0.37f1	 6000.0.37f1	 6000.0.37f1	 6000.0.37f1

Content

- [Unity Image Loader](#)
 - [Features](#)
 - [Stability status](#)
 - [Content](#)
 - [Installation](#)
 - [Option 1 - Installer](#)
 - [Option 2 - OpenUPM-CLI](#)
 - [Option 3 - Asset Store](#)
- [Usage](#)
 - [Events lifecycle](#)
 - [Load Sprite then set into Image](#)
 - [Load Texture2D then set into Material](#)
 - [Load Sprite then set into multiple Image](#)
 - [Error handling](#)
 - [Async await and Forget](#)
 - [Placeholder](#)
 - [Cancellation](#)
 - [Cancel by MonoBehaviour events](#)
 - [Explicit cancellation](#)
 - [Cancellation Token](#)
 - [Cancellation by using](#)
 - [Timeout](#)
 - [Cache](#)
 - [Setup Cache](#)
 - [Change Disk cache folder](#)
 - [Override for a specific loading task](#)

- Manually read / write into cache
 - Check cache existence
 - Clear cache
 - Texture Memory Management
 - Manual Memory cache cleaning
 - Automatic Memory cache cleaning
 - Load Reference
 - Dispose `Reference<T>` on `Component` destroy event
 - Get references count
 - Other
 - Understanding `IFuture<T>`
 - Key Properties of `IFuture<T>`
 - Key Methods of `IFuture<T>`
 - Example Usage of `IFuture<T>`
 - Understanding `Reference<T>`
 - Key Properties of `Reference<T>`
 - Key Methods of `Reference<T>`
 - Example Usage of `Reference<T>`
 - Understanding `Future<Reference<T>>`
 - Why It Is Needed
-

Installation

- [Install OpenUPM-CLI](#)
- Open command line in Unity project folder
- Run the command

```
openupm add extensions.unity.imageloader
```

Option 1 - Installer

- [!\[\]\(67ff022fd78f943b679992c2874bbfd1_img.jpg\) Download Installer](#)
-  **Import installer into Unity project**
 - You may use double click on the file - Unity will open it
 - OR: You may open Unity Editor first, then click on `Assets/Import Package/Custom Package`, then choose the file

Option 2 - OpenUPM-CLI

- [!\[\]\(2a133ebb0337313d16cc068f19494aa2_img.jpg\) Install OpenUPM-CLI](#)
-  Open command line in Unity project folder

```
openupm add extensions.unity.imageloader
```

- [▶ Open Asset Store](#)

In the main thread somewhere at the start of the project need to call `ImageLoader.Init()`; once to initialize static properties in the right thread. It is required to make in the main thread. Then you can use `ImageLoader` from any thread and at any time.

Full sample source code

```
ImageLoader.LoadSprite(imageURL) // loading process started  
//  
  
    | Loading lifecycle events |  
|  
// _____|  
|  
.LoadedFromMemoryCache(sprite => Debug.Log("Loaded from memory cache")) // on loaded from memory cache |  
.LoadingFromDiskCache (( )=> Debug.Log("Loading from disk cache")) // on loading from disk cache |  
.LoadedFromDiskCache (sprite => Debug.Log("Loaded from disk cache")) // on loaded from disk cache |  
.LoadingFromSource (( )=> Debug.Log("Loading from source")) // on loading from source |  
.LoadedFromSource (sprite => Debug.Log("Loaded from source")) // on loaded from source |  
//  
_____  
  
// Success lifecycle events  
// _____|  
|.Loaded(sprite => Debug.Log("Loaded")) // on successfully loaded  
//  
// Set/Consume sprite [placeholder, successfully loaded]
```

```

//
.Consume(sprite => Debug.Log("Consumed")) //
.Consume(image) //
//

//
// Negative lifecycle events
//
.Canceled(() => Debug.Log("Canceled")) // on canceled
.Failed(exception => Debug.LogException(exception)) // on failed to load
//

//

// | The end of lifecycle |
//

.Completed(isLoaded => Debug.Log($"Completed, isLoaded={isLoaded}")) // on
completed //
[loaded, failed or canceled] //

.Forget(); // removes the compilation warning, does nothing else

```

Load **Sprite** then set into **Image**

[Full sample source code](#)

```

// Load a sprite from the web and cache it for faster loading next time
image.sprite = await ImageLoader.LoadSprite(imageURL);

// Load a sprite from the web and set it directly to the Image component
await ImageLoader.LoadSprite(imageURL).Consume(image);

```

Load **Texture2D** then set into **Material**

[Full sample source code](#)

```

// Load a Texture2D from the web and cache it for faster loading next time
material.mainTexture = await ImageLoader.LoadTexture(imageURL);

// Load a Texture2D from the web and set it directly to the Material
await ImageLoader.LoadTexture(imageURL).Consume(material);

```

Load **Sprite** then set into multiple **Image**

[Full sample source code](#)

```
ImageLoader.LoadSprite(imageURL).Consume(image1, image2).Forget();
```

Error handling

[Full sample source code](#)

```
ImageLoader.LoadSprite(imageURL) // Attempt to load a sprite
    .Consume(image) // If successful, set the sprite to the Image component
    .Failed(exception => Debug.LogException(exception)) // If an error occurs, log
the exception
    .Forget(); // Forget the task to avoid compilation warning

ImageLoader.LoadSprite(imageURL) // Attempt to load a sprite
    .Consume(image) // If successful, set the sprite to the Image component
    .Then(sprite => image.gameObject.SetActive(true)) // If successful, activate
the GameObject
    .Failed(exception => image.gameObject.SetActive(false)) // If an error occurs,
deactivate the GameObject
    .Forget(); // Forget the task to avoid compilation warning
```

Async **await** and **Forget**

[Full sample source code](#)

```
// Load image and wait
await ImageLoader.LoadSprite(imageURL);

// Load image, set image and wait
await ImageLoader.LoadSprite(imageURL).Consume(image);

// Skip waiting for completion.
// To do that we can simply remove 'await' from the start.
// To avoid compilation warning need to add '.Forget()'.
ImageLoader.LoadSprite(imageURL).Consume(image).Forget();
```

Placeholder

While the target image is loading it would be a good idea to set placeholder image. Also, it works well for setting image if loading fails.

[Full sample source code](#)

```
ImageLoader.LoadSprite(imageURL)
    // set placeholder in all conditions
    .SetPlaceholder(placeholderAny)

    // set placeholder in a specific conditions
    .SetPlaceholder(placeholderLoadingFromSource,
PlaceholderTrigger.LoadingFromSource)
    .SetPlaceholder(placeholderFailedToLoad, PlaceholderTrigger.FailedToLoad)

    // set consumer
    .Consume(image)
    .Forget();
```

Cancellation

Cancellation is helpful if target image consumer doesn't exist anymore. For example the `Image` was destroyed because another level had been loaded. There is not much sense to continue to load the image. It would save some network traffic, CPU resources, and RAM. `IFuture<T>` provides a wide range of options to cancel the ongoing loading process.

[Full sample source code](#)

Cancel by MonoBehaviour events

```
ImageLoader.LoadSprite(imageURL)
    .Consume(image)
    .CancelOnEnable(this)    // cancel on OnEnable event of current MonoBehaviour
    .CancelOnDisable(this)  // cancel on OnDisable event of current MonoBehaviour
    .CancelOnDestroy(this); // cancel on OnDestroy event of current MonoBehaviour
```

Explicit cancellation

```
var future = ImageLoader.LoadSprite(imageURL).Consume(image);
future.Cancel();
```

Cancellation Token

```
var cancellationTokenSource = new CancellationTokenSource();

// loading with attached cancellation token
ImageLoader.LoadSprite(imageURL, cancellationToken: cancellationTokenSource.Token)
    .Consume(image)
    .Forget();

cancellationTokenSource.Cancel(); // canceling
```

```
var cancellationTokenSource = new CancellationTokenSource();

ImageLoader.LoadSprite(imageURL)
    .Consume(image)
    .Register(cancellationTokenSource.Token) // registering cancellation token
    .Forget();

cancellationTokenSource.Cancel(); // canceling
```

Cancellation by **using**

```
using (var future = ImageLoader.LoadSprite(imageURL).Consume(image))
{
    // future would be canceled and disposed outside of the brackets
}
```

```
ImageLoader.LoadSprite(imageURL) // load sprite
    .Consume(image) // if success set sprite into image
    .CancelOnDestroy(this) // cancel OnDestroy event of current gameObject
    .Forget();

ImageLoader.LoadSprite(imageURL) // load sprite
    .Consume(image) // if success set sprite into image
    .Failed(exception => Debug.LogException(exception)) // if fail print exception
    .CancelOnDestroy(this) // cancel OnDestroy event of current gameObject
    .Forget();

ImageLoader.LoadSprite(imageURL) // load sprite
    .Consume(image) // if success set sprite into image
    .Then(sprite => image.gameObject.SetActive(true)) // if success activate
gameObject
    .Failed(exception => image.gameObject.SetActive(false)) // if fail deactivate
gameObject
    .Canceled(() => Debug.Log("ImageLoading canceled")) // if cancelled
    .CancelOnDisable(this) // cancel OnDisable event of current gameObject
    .Forget();
```

Timeout

Timeout triggers **IFuture<T>** cancellation.

[Full sample source code](#)

Set global timeout in the settings:


```
ImageLoader.settings.timeout = TimeSpan.FromSeconds(30);
```

Set timeout for a specific loading request (`IFuture<T>`):

```
ImageLoader.LoadSprite(imageURL) // load sprite
    .Consume(image) // if success set sprite into image
    .Timeout(TimeSpan.FromSeconds(10)) // set timeout duration 10 seconds
    .Forget();
```

Cache

Cache system based on the two layers. The first layer is **Memory cache**, second is **Disk cache**. Each layer could be enabled or disabled. Could be used without caching at all. By default both layers are enabled. `WebGL` doesn't support Disk cache, because it doesn't have access to disk.

Setup Cache

- `ImageLoader.settings.useMemoryCache = true;` default value is `true`
- `ImageLoader.settings.useDiskCache = true;` default value is `true`

Change Disk cache folder

By default it uses `Application.persistentDataPath + "/ImageLoader"`

```
ImageLoader.settings.diskSaveLocation = Application.persistentDataPath +
    "/myCustomFolder";
```

Override for a specific loading task

It overrides global `ImageLoader.settings`

```
ImageLoader.LoadSprite(url)
    .SetUseDiskCache(false)
    .SetUseMemoryCache(true);
```

[Full sample source code](#)

Manually read / write into cache

```
// Override Memory cache for specific image
ImageLoader.SaveToMemoryCache(url, sprite);
```

```
// Take from Memory cache for specific image if exists
ImageLoader.LoadSpriteFromMemoryCache(url);
```

Check cache existence

```
// Check if any cache contains specific image
ImageLoader.CacheContains(url);

// Check if Memory cache contains specific image
ImageLoader.MemoryCacheContains(url);

// Check if Disk cache contains specific image
ImageLoader.DiskCacheContains(url);
```

Clear cache

```
// Clear memory Memory and Disk cache for all images
ImageLoader.ClearCacheAll();

// Clear only Memory and Disk cache for specific image
ImageLoader.ClearCache(url);

// Clear only Memory cache for all images
ImageLoader.ClearMemoryCacheAll();

// Clear only Memory cache for specific image
ImageLoader.ClearMemoryCache(url);

// Clear only Disk cache for all images
ImageLoader.ClearDiskCacheAll();

// Clear only Disk cache for specific image
ImageLoader.ClearDiskCache(url);
```

Memory cache could be cleared automatically if to use [Reference<T>](#) and the heavy [Texture2D](#) memory would be released as well. Read more at [Texture Memory Management](#).

Texture Memory Management

Texture2D objects consume a lot of memory. Ignoring it may impact performance or even trigger [OutOfMemory](#) crash by operation system. To avoid it, let's dig deeper into tools the package provides. We worry less about Disk cache, because it doesn't impact game performance directly. Let's focus on the Memory cache.

Manual Memory cache cleaning

It is simple, just executing this line of code would release memory of a single Texture2D in the case if no other **Reference** pointing on it exists. Before doing that please make sure that no Unity component is using the texture.

```
ImageLoader.ClearMemoryCache(url);
```

Under the hood it calls **UnityEngine.Object.DestroyImmediate(texture)**.

 Releasing **Texture2D** from memory while any Unity's component uses it may trigger native app crash or even Unity Editor crash

Automatic Memory cache cleaning

ImageLoader can manager memory releasing of loaded textures. To use it need to call **ImageLoader.LoadSpriteRef** instead of **ImageLoader.LoadSprite**. It returns **Reference<Sprite>** object which contains **Sprite** and **Url**. When **Reference<Sprite>** object is not needed anymore, call **reference.Dispose()** method to release memory, or just don't save the reference on it. It is **IDisposable** and it will be disposed by Garbage Collector. Each new instance of **Reference<Sprite>** increments reference counter of the texture. When the last reference is disposed, the texture memory releases. Also, if any reference is alive, calling **ImageLoader.ClearMemoryCache** or **ImageLoader.ClearCache** would have zero effect for only referenced textures. It prints warning messages about it.

```
// Load sprite image and get reference to it
var reference = await ImageLoader.LoadSpriteRef(imageURL);

// Take from Memory cache reference for specific image if exists
var reference = ImageLoader.LoadSpriteRefFromMemoryCache(url);

// Dispose `reference` when you don't need the texture anymore
reference.Dispose();

// You may also nullify the reference to let Garbage Collector at some point to
Dispose it for you
reference = null;
```

 Releasing **Texture2D** from memory while any Unity's component uses it may trigger native app crash or even Unity Editor crash. Please pay enough attention to manage **Reference<T>** instances in a proper way. Or do not use them.

Load Reference

[Full sample source code](#)

Future<Reference<T>>.Consume has a unique feature to attach the reference to the target consumer if consumer is **UnityEngine.Component**. The reference would be disposed as only the consumer gets destroyed.

```
ImageLoader.LoadSpriteRef(imageURL) // load sprite using Reference
    .Consume(image) // if success set sprite into image, also creates binding to
`image`
    .Forget();
```

Dispose **Reference<T>** on **Component** destroy event

It automatically dispose the reference as only **this.gameObject** gets **OnDestroy** callback.

```
ImageLoader.LoadSpriteRef(imageURL) // load sprite using Reference
    .Then(reference => reference.DisposeOnDestroy(this))
    .Then(reference =>
    {
        var sprite = reference.Value;
        // use sprite
    })
    .Forget();
```

Get references count

```
var count = ImageLoader.GetReferenceCount(imageURL); // get count of references
```

Other

Understanding **IFuture<T>**

The **IFuture<T>** interface represents an asynchronous operation that will eventually produce a result of type **T**. It provides a range of methods and properties to handle the lifecycle of the asynchronous operation, including loading, success, failure, and cancellation events.

Key Properties of **IFuture<T>**

- **Id**: Unique identifier for the future.
- **Url**: URL associated with the future.
- **IsCancelled**: Indicates if the operation has been cancelled.
- **IsLoaded**: Indicates if the operation has successfully loaded the result.
- **IsCompleted**: Indicates if the operation has completed (either successfully or with an error).
- **IsInProgress**: Indicates if the operation is currently in progress.
- **Status**: Current status of the future.
- **CancellationToken**: Token used to cancel the operation.
- **Value**: The result of the operation.
- **LogLevel**: The logging level for the operation.

Key Methods of `IFuture<T>`

- **`Then(Action<T> onCompleted)`**: Registers a callback to be executed when the operation successfully completes and produces a result.
- **`Failed(Action<Exception> action)`**: Registers a callback to be executed if the operation fails with an exception.
- **`Completed(Action<bool> action)`**: Registers a callback to be executed when the operation completes, regardless of success or failure. The boolean parameter indicates whether the operation was successful.
- **`Canceled(Action action)`**: Registers a callback to be executed if the operation is canceled.
- **`SetUseDiskCache(bool value = true)`**: Configures whether the operation should use disk caching.
- **`SetUseMemoryCache(bool value = true)`**: Configures whether the operation should use memory caching.
- **`SetLogLevel(DebugLevel value)`**: Sets the logging level for the operation.
- **`Cancel()`**: Cancels the operation if it is still in progress.
- **`Forget()`**: Ignores the result of the operation, useful for avoiding compilation warnings about unawaited tasks.
- **`AsUniTask()`**: Converts the `IFuture<T>` instance to a `UniTask<T>`.
- **`AsTask()`**: Converts the `IFuture<T>` instance to a `Task<T>`.
- **`AsReference(DebugLevel logLevel = DebugLevel.Trace)`**: Converts the `IFuture<T>` instance to a `Future<Reference<T>>` instance.
- **`GetAwaiter()`**: Returns an awaiter for the `IFuture<T>` instance, allowing it to be awaited using the `await` keyword.
- **`PassEvents(IFutureInternal<T> to, bool passCancelled = true)`**: Passes events to another future.
- **`PassEvents<T2>(IFutureInternal<T2> to, Func<T, T2> convert, bool passCancelled = true)`**: Passes events to another future with conversion.
- **`Register(Cancellation_token cancellationToken)`**: Registers a new cancellation token to cancel the future with it.
- **`Timeout(TimeSpan duration)`**: Sets a timeout duration for the future. If the duration is reached, it fails the future with a related exception.

Example Usage of `IFuture<T>`

```
ImageLoader.LoadSprite(imageURL) // Start loading the sprite
    .Then(sprite => Debug.Log("Loaded")) // On successful load
    .Failed(exception => Debug.LogException(exception)) // On failure
    .Completed(isLoaded => Debug.Log($"Completed, isLoaded={isLoaded}")) // On
completion
    .Canceled(() => Debug.Log("Canceled")) // On cancellation
    .Forget(); // Avoid compilation warnings
```

Understanding `Reference<T>`

The `Reference<T>` class is used to manage the lifecycle of loaded resources, such as `Texture2D` or `Sprite`, in a memory-efficient manner. It helps to automatically release memory when the resource is no longer

needed, preventing memory leaks and optimizing performance.

Key Properties of `Reference<T>`

- **Value**: The actual resource (e.g., `Sprite` or `Texture2D`) that is being referenced.
- **Url**: The URL associated with the resource.
- **IsDisposed**: Indicates whether the reference has been disposed.

Key Methods of `Reference<T>`

- **Dispose()**: Disposes the reference, releasing the associated resource from memory. This should be called when the resource is no longer needed.
- **DisposeOnDestroy(Component component)**: Automatically disposes the reference when the specified `Component` is destroyed. This is useful for ensuring that resources are released when the associated `GameObject` is destroyed.
- **DisposeOnDisable(Component component)**: Automatically disposes the reference when the specified `Component` is disabled.
- **DisposeOnEnable(Component component)**: Automatically disposes the reference when the specified `Component` is enabled.

Example Usage of `Reference<T>`

```
// Load a sprite using Reference
var reference = await ImageLoader.LoadSpriteRef(imageURL);

// Use the sprite
var sprite = reference.Value;

// Dispose the reference when the sprite is no longer needed
reference.Dispose();
```

```
// Alternatively, automatically dispose the reference when the GameObject is
// destroyed
ImageLoader.LoadSpriteRef(imageURL)
    .Then(reference => reference.DisposeOnDestroy(this))
    .Then(reference =>
    {
        var sprite = reference.Value;
        // use sprite
    })
    .Forget();
```

Understanding `Future<Reference<T>>`

The `Future<Reference<T>>` class combines the functionality of `IFuture<T>` and `Reference<T>`, providing a powerful tool for managing the lifecycle of asynchronous operations that produce resources such as

`Texture2D` or `Sprite`. This combination allows you to handle the loading process and the resource management in a memory-efficient manner.

Why It Is Needed

The `Future<Reference<T>>` class is needed to ensure that resources are loaded asynchronously and managed efficiently. By using this class, you can:

1. **Load Resources Asynchronously:** Start loading resources such as `Texture2D` or `Sprite` without blocking the main thread.
2. **Manage Resource Lifecycle:** Automatically release resources when they are no longer needed, preventing memory leaks and optimizing performance.
3. **Handle Loading Events:** Register callbacks for various events such as success, failure, and cancellation during the loading process.
4. **Bind Resources to Components:** Automatically dispose of resources when the associated `UnityEngine.Component` is destroyed, ensuring that resources are released when the `GameObject` is destroyed.